

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Dot .Net Core and Progressive Web Application Practical

Practical – V

Roll No.:T006	Name:Nandini Chaudhari
Class:TYIT	Batch:01
Date of Assignment:28-07-2026	Date/Time of Submission:28-07-2026

a) Create a web application to demonstrate the use of different types of Cookies.

Algorithm:

Step1:- Start the program

Step2:- application and load WebForm1. Initialize the ViewState value on the first page load.

Step3:- CheckPostBack: If the page is reloaded, update and display the ViewState count; otherwise, set the initial count to 1.

Step4:- On Button1 Click, retrieve the value stored in ViewState and display it in Label1.

Step5:- On Button2 Click, create a Cookie named "name", store the textbox value in it, and redirect the user to WebForm2.

Step6:- In WebForm2, read the cookie value and display a Welcome message if the cookie exists, then

End.

Code:

webform1:-

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;
namespace prac5
{
    public partial class WebForm1 : System.Web.UI.Page
    {
        protected void Page_Load(object sender, EventArgs e)
        {
            if(IsPostBack)
            {
                int ViewStateVal = Convert.ToInt32(ViewState["count"]) + 1;
                Label2.Text = "View State = " + ViewStateVal.ToString();
            }
            else
            {
                ViewState["count"] = 1;
            }
        }
        protected void Button1_Click(object sender, EventArgs e)
        {
            Label1.Text = ViewState["count"].ToString();
        }
        protected void Button2_Click(object sender, EventArgs e)
        {
            HttpCookie h = new HttpCookie("name");
            h.Value = TextBox1.Text;
            Response.Cookies.Add(h);
            Response.Redirect("webForm2.aspx");
        }
    }
}
```

```

    }
}
}
Web form2
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;
namespace prac5
{
    public partial class WebForm2 : System.Web.UI.Page
    {
        protected void Page_Load(object sender, EventArgs e)
        {
            if (Request.Cookies["name"] != null)
            {
                Response.Write("Welcome " + Request.Cookies["name"].Value);
            }
        }
    }
}

```

Label

Label

View State

Cookies

Output:

1

View State = 2

View State

Nandini T006

Cookies

Welcome Nandini T006

b) Create a Product Table in SQL Server and perform Insert Operation using Entity Framework Core.

Algorithm:

Step1:- Create Project: Open Visual Studio Create new ASP.NET Core Web App (MVC).

Step2:- Install Packages: Open Package Manager Console and run the necessary Entity Framework

Core commands.

Step3:- Install-Package Microsoft.EntityFrameworkCore.SqlServer

Step4:- Install-Package Microsoft.EntityFrameworkCore.Tools

Step5:- Define Model & DbContext: Create Product.cs (Id, Name, Price) and AppDbContext.cs

containing DbSet<Product> Products.

Step6:-Configure Database Connection: Add connection string in appsettings.json and register AddDbContext in Program.cs.

AddDbContext in Program.cs.

Code:

Step 1: Product Model ([Product.cs](#))

```
1 namespace ProductEFCore.Models
2 {
3     3 references
4     public class Product
5     {
6         0 references
7         public int Id { get; set; }
8         1 reference
9         public string Name { get; set; }
10        1 reference
11        public int Price { get; set; }
12    }
13 }
```

Step 2: AppDbContext ([AppDbContext.cs](#))

```
using Microsoft.EntityFrameworkCore;
using ProductEFCore.Models;
7 references
public class AppDbContext: DbContext
{
    0 references
    public AppDbContext(DbContextOptions<AppDbContext> options) : base (options)
    {
    }
    1 reference
    public DbSet<Product> Products { get; set; }
}
```

Step 3: appsettings.json

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "DefaultConnection": "Server=(localdb)\\MSSQLLocalDB;Database=ProductDB;Trusted_Connection=True"
  }
}
```

Step 4: [Program.cs](#)

```
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

builder.Services.AddDbContext<AppDbContext>(options =>
options.UseSqlServer(
builder.Configuration.GetConnectionString("DefaultConnection")));

var app = builder.Build();

app.UseStaticFiles();

app.UseRouting();

app.MapControllerRoute(
name: "default",
pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

Step 6: HomeController

```
1 reference
public class HomeController : Controller
{
    private readonly AppDbContext _context;
    0 references
    public HomeController(AppDbContext context)
    {
        _context = context;
    }
    0 references
    public IActionResult Index()
    {
        Product p = new Product();
        p.Name = "Laptop";
        p.Price = 50000;
        _context.Products.Add(p);
        _context.SaveChanges();
        return Content("Product Inserted Successfully");
    }
}
```

Output:

Product Inserted Successfully

```
1 select * from products;
```

	Id	Name	Price
1	1	Laptop	50000

c) Develop a web application to perform CRUD Operations (Create, Read, Update, Delete) on the Employee table using EF Core.

Algorithm:

Step1:-Define Model: Create Employee.cs with properties (Id, Name, Department, Salary).

Step2:-Setup DbContext: Create ApplicationDbContext.cs with DbSet<Employee> Employees and register it inside Program.cs.

Step3:- Apply Migrations: Run Add-Migration AddEmployeeTable followed by Update-Database in Package Manager Console.

Step4:-Build Controller Actions: In EmployeesController.cs, implement methods for Create, Read, Update, and Delete operations.

Step5:-Create Views: Generate corresponding Razor views under Views/Employees/ (Index, Create, Edit, Delete).

Code:

Employee.cs

```
namespace EmployeeCRUD.Models
{
    public class Employee
    {
        public int Id { get; set; }
        public required string Name { get; set; }

        public required string Department { get; set; }

        public double Salary { get; set; }
    }
}
```

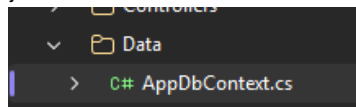
Program.cs

```
using Microsoft.EntityFrameworkCore;
using EmployeeCRUD.Data;
builder.Services.AddControllersWithViews();
builder.Services.AddDbContext<AppDbContext> (options =>
options.UseSqlServer(builder.Configuration.GetConnectionString("MyConnection")));
appsettings.json:
"AllowedHosts": "*",
"ConnectionStrings": {
    "MyConnection":
    "Server=(localdb)\\MSSQLLocalDB;Database=EmployeeDB;Trusted_Connection=True;TrustServerCertificate=True",
```

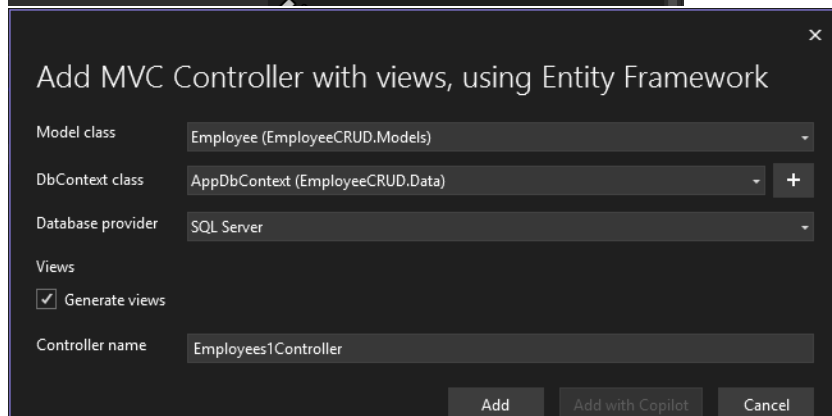
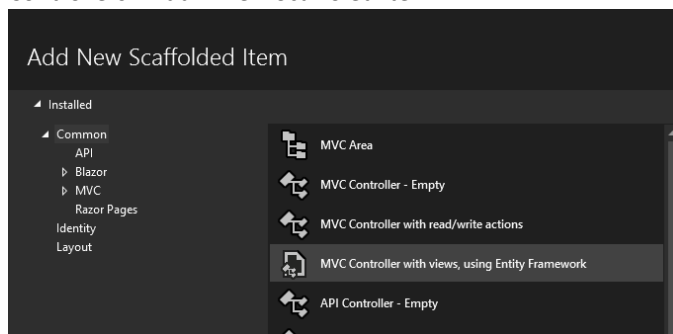
```

"AppDbContext":
"Server=(localdb)\\mssqllocaldb;Database=AppDbContext;Trusted_Connection=True;MultipleActiveResult
Sets=true"
}
AppDbContext.cs
using Microsoft.EntityFrameworkCore;
using EmployeeCRUD.Models;
namespace EmployeeCRUD.Data
{
    public class AppDbContext:DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) {}
        public DbSet <Employee> Employees { get; set; }
    }
}

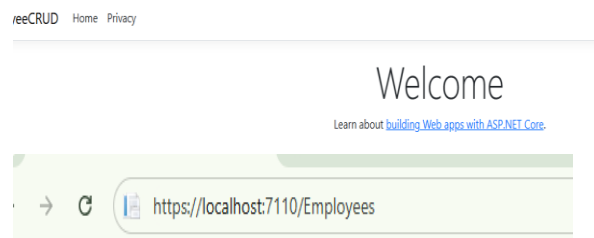
```



Controllers->Add-> New Scaffolded Item



Output:



EmployeeCRUD Home Privacy

EmployeeCRUD Home Privacy

Create Employee

Name

Department

Salary

[Create](#)

[Back to List](#)

EmployeeCRUD Home Privacy

Index

[Create New](#)

Name	Department	Salary	
Nandini	IT	90000	Edit Details Delete

d) Authentication and Authorization in ASP.NET Core.

Algorithm:

Step1:-Register Auth Services: In Program.cs, add services using `AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme).AddCookie(...)` and `AddAuthorization()`.

Step2:-Configure Pipeline: Add `app.UseAuthentication()` before `app.UseAuthorization()` in Program.cs.

Step3:-Build Login Logic: In HomeController.cs, validate user credentials, create a ClaimsIdentity, and call `await HttpContext.SignInAsync(...)`.

Step4:-Protect Route: Add the `[Authorize]` attribute directly above the `Secure()` action method.

Code:

program.cs

```
using Microsoft.AspNetCore.Authentication.Cookies;
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllersWithViews();
builder.Services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)
    .AddCookie(options =>
    {
        options.LoginPath = "/Home/Index";
    });
builder.Services.AddAuthorization();
var app = builder.Build();
app.UseStaticFiles();
app.UseRouting();
app.UseAuthentication();
app.UseAuthorization();
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
app.Run();
```

homecontroller.cs

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Authentication;
using Microsoft.AspNetCore.Authentication.Cookies;
using System.Security.Claims;
namespace AuthDemo.Controllers
{
    0 references
    public class HomeController : Controller
    {
        0 references
        public IActionResult Index()
        {
            return View();
        }
        [HttpPost]
```

```

public async Task<IActionResult> Index(string username, string password)
{
    if (username == "Nandini" && password == "2006")
    {
        var claims = new List<Claim>
        {
            new Claim(ClaimTypes.Name, username)
        };
        var identity = new ClaimsIdentity(claims,
            CookieAuthenticationDefaults.AuthenticationScheme);
        await HttpContext.SignInAsync(
            CookieAuthenticationDefaults.AuthenticationScheme,
            new ClaimsPrincipal(identity));
        return RedirectToAction("Secure");
    }
    ViewBag.Message = "Invalid Username or Password";
    return View();
}

[Authorize]

public IActionResult Secure()
{
    return Content("Welcome! Login Successful.");
}

0 references
public async Task<IActionResult> Logout()
{
    await HttpContext.SignOutAsync(
        CookieAuthenticationDefaults.AuthenticationScheme);
    return RedirectToAction("Index");
}
}

```

index.cshtml

```

<h2>Login</h2>
<form method="post">
    Username :
    <input type="text" name="username" /><br /><br />
    Password :
    <input type="password" name="password" /><br /><br />
    <input type="submit" value="Login" />
</form>
<p style="color: red">
    @ViewBag.Message
</p>

```

Output:

Login

Username :

Password : 

Welcome! Login Successful.